Instituto tecnológico de Tijuana

Ing. En sistemas computacionales

Proyecto / Tarea / Práctica:

Tarea:

Actividad de evidencia Individual del patrón: Prototipo

Materia:

Patrones de diseño

Unidad:

Unidad 2

Facilitador:

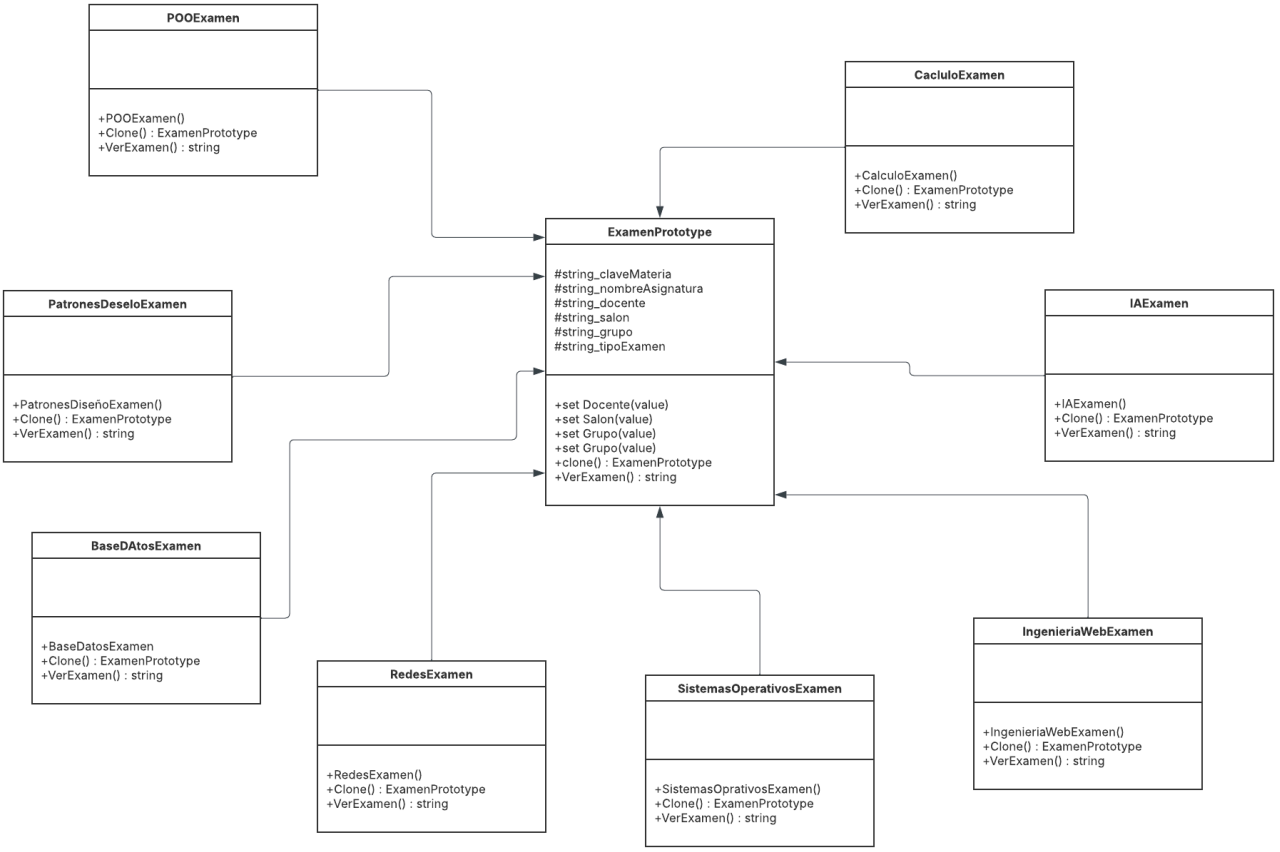
MARIBEL GUERRERO LUIS

Alumno:

MOJICA FAJARDO JOSE ANGEL - 22210322

Fecha:
Tijuana Baja California a 10 de Marzo del 2026

Diagrama UML



Código

```
namespace PatronPrototipoExamenes
{
    class Program
    {
        static void Main(string[] args)
        {
            // Cada materia tiene su propio prototipo con datos fijos
            ExamenPrototype prototipoPatrones = new PatronesDisenioExamen();
            ExamenPrototype prototipoPOO = new POOExamen();
            ExamenPrototype prototipoBD = new BaseDatosExamen();
            ExamenPrototype prototipoRedes = new RedesExamen();
            ExamenPrototype prototipoSO = new SistemasOperativosExamen();
            ExamenPrototype prototipoWeb = new IngenieriaWebExamen();
            ExamenPrototype prototipoCalculo = new Calculo1Examen();
            ExamenPrototype prototipoIA = new IAExamen();

                // Esto es lo que se colna del prototipo y solo se cambia lo que varía por
grupo
            ExamenPrototype patronesG1 = prototipoPatrones.Clone();
            patronesG1.Grupo = "GDS-81A";
            patronesG1.Salon = "Aula 301";
            patronesG1.TipoExamen = "Parcial 1";
            Console.WriteLine(patronesG1.VerExamen());


            ExamenPrototype patronesG2 = prototipoPatrones.Clone();
            patronesG2.Docente = "Mtra. Lopez";
            patronesG2.Grupo = "GDS-81B";
            patronesG2.Salon = "Aula 302";
            patronesG2.TipoExamen = "Final";
            Console.WriteLine(patronesG2.VerExamen());

            ExamenPrototype pooG1 = prototipoPOO.Clone();
```

```
pooG1.Grupo = "GDS-61A";
pooG1.Salon = "Lab 1";
pooG1.TipoExamen = "Parcial 2";
Console.WriteLine(pooG1.VerExamen());

ExamenPrototype bdG1 = prototipoBD.Clone();
bdG1.Grupo = "GDS-61A";
bdG1.Salon = "Aula 110";
bdG1.TipoExamen = "Parcial 1";
Console.WriteLine(bdG1.VerExamen());

ExamenPrototype redesG1 = prototipoRedes.Clone();
redesG1.Grupo = "GDS-71A";
redesG1.Salon = "Lab Redes";
redesG1.TipoExamen = "Final";
Console.WriteLine(redesG1.VerExamen());

ExamenPrototype soG1 = prototipoSO.Clone();
soG1.Grupo = "GDS-71A";
soG1.Salon = "Aula 210";
soG1.TipoExamen = "Parcial 1";
Console.WriteLine(soG1.VerExamen());

ExamenPrototype webG1 = prototipoWeb.Clone();
webG1.Grupo = "GDS-81D";
webG1.Salon = "Lab Web";
webG1.TipoExamen = "Parcial 2";
Console.WriteLine(webG1.VerExamen());

ExamenPrototype calG1 = prototipoCalculo.Clone();
calG1.Grupo = "GDS-11A";
calG1.Salon = "Aula 101";
calG1.TipoExamen = "Parcial 1";
Console.WriteLine(calG1.VerExamen());
```

```
            ExamenPrototype iaG1 = prototipoIA.Clone();
            iaG1.Docente = "Dr. Herrera";
            iaG1.Grupo = "GDS-91A";
            iaG1.Salon = "Lab IA";
            iaG1.TipoExamen = "Final";
            Console.WriteLine(iaG1.VerExamen());

            Console.ReadKey();
        }
    }
}

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace PatronPrototipoExamenes
{

    // Clase base del patron Prototype.
    // HERENCIA: todas las materias se basan en esta clase,
    // lo que significa que comparten los mismos atributos y metodos.
    //
    // DATOS PROTEGIDOS: los atributos usan "protected", esto significa
    // que solo esta clase y las que se basen en ella pueden usarlos,
    // nadie mas puede modificarlos directamente desde afuera.
    // CLONACION: obliga a cada materia a definir su propio Clone().

    public abstract class ExamenPrototype
    {
```

```
        // Estos son los datos protegidos, no son publicos, cada materia los define en su constructor
        protected string _claveMateria;
        protected string _nombreAsignatura;
        protected string _docente;
        protected string _salon;
        protected string _grupo;
        protected string _tipoExamen;

        // Solo estos datos se pueden cambiar desde fuera, son los que varian por grupo
        public string Docente { set => _docente = value; }
        public string Salon { set => _salon = value; }
        public string Grupo { set => _grupo = value; }
        public string TipoExamen { set => _tipoExamen = value; }

        // Cada materia define como se clona y como se muestra en pantalla
        public abstract ExamenPrototype Clone();
        public abstract string VerExamen();
    }
}



using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace PatronPrototipoExamenes
{
    public class IAExamen : ExamenPrototype
    {
        public IAExamen()
```

```
        {
            _claveMateria = "TIC-420";
            _nombreAsignatura = "Inteligencia Artificial";
            _docente = "Dr. Ramirez";
        }
                                public     override     ExamenPrototype     Clone()     =>
(IAExamen)this.MemberwiseClone();
        public override string VerExamen() => $"{_nombreAsignatura} [{_claveMateria}]
Docente: {_docente} Grupo: {_grupo} Salon: {_salon} Tipo: {_tipoExamen}";
    }
}


using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace PatronPrototipoExamenes
{
    public class Calculo1Examen : ExamenPrototype
    {
        public Calculo1Examen()
        {
            _claveMateria = "MAT-101";
            _nombreAsignatura = "Calculo Diferencial";
            _docente = "Dr. Herrera";
        }
                                public     override     ExamenPrototype     Clone()     =>
(Calculo1Examen)this.MemberwiseClone();
        public override string VerExamen() => $"{_nombreAsignatura} [{_claveMateria}]
Docente: {_docente} Grupo: {_grupo} Salon: {_salon} Tipo: {_tipoExamen}";
    }
```

```
}


using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace PatronPrototipoExamenes
{
    public class IngenieriaWebExamen : ExamenPrototype
    {
        public IngenieriaWebExamen()
        {
            _claveMateria = "TIC-315";
            _nombreAsignatura = "Ingenieria Web";
            _docente = "Mtra. Lopez";
        }
                                    public    override    ExamenPrototype    Clone()    =>
(IngenieriaWebExamen)this.MemberwiseClone();
         public override string VerExamen() => $"{_nombreAsignatura} [{_claveMateria}]
Docente: {_docente} Grupo: {_grupo} Salon: {_salon} Tipo: {_tipoExamen}";
    }
}




using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
```

```
namespace PatronPrototipoExamenes
{
    public class SistemasOperativosExamen : ExamenPrototype
    {
        public SistemasOperativosExamen()
        {
            _claveMateria = "TIC-308";
            _nombreAsignatura = "Sistemas Operativos";
            _docente = "Ing. Vega";
        }
                                    public    override    ExamenPrototype    Clone()    =>
(SistemasOperativosExamen)this.MemberwiseClone();
         public override string VerExamen() => $"{_nombreAsignatura} [{_claveMateria}]
Docente: {_docente} Grupo: {_grupo} Salon: {_salon} Tipo: {_tipoExamen}";
    }
}


using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace PatronPrototipoExamenes
{
    public class RedesExamen : ExamenPrototype
    {
        public RedesExamen()
        {
            _claveMateria = "TIC-310";
            _nombreAsignatura = "Redes de Computadoras";
            _docente = "Mtra. Lopez";
```

```
        }
                                public     override     ExamenPrototype     Clone()     =>
(RedesExamen)this.MemberwiseClone();
        public override string VerExamen() => $"{_nombreAsignatura} [{_claveMateria}]
Docente: {_docente} Grupo: {_grupo} Salon: {_salon} Tipo: {_tipoExamen}";
    }
}


using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace PatronPrototipoExamenes
{
    public class BaseDatosExamen : ExamenPrototype
    {
        public BaseDatosExamen()
        {
            _claveMateria = "TIC-305";
            _nombreAsignatura = "Base de Datos";
            _docente = "Dr. Ramirez";
        }
                                public     override     ExamenPrototype     Clone()     =>
(BaseDatosExamen)this.MemberwiseClone();
        public override string VerExamen() => $"{_nombreAsignatura} [{_claveMateria}]
Docente: {_docente} Grupo: {_grupo} Salon: {_salon} Tipo: {_tipoExamen}";
    }
}

using System;
using System.Collections.Generic;
using System.Linq;
```

```
using System.Text;
using System.Threading.Tasks;

namespace PatronPrototipoExamenes
{
    // Materia: Programacion Orientada a Objetos.
    // HERENCIA: toma todos los atributos de ExamenPrototype.
    // CLONACION: MemberwiseClone hace una copia exacta de este examen.
    public class POOExamen : ExamenPrototype
    {
        public POOExamen()
        {
            _claveMateria = "TIC-302";
            _nombreAsignatura = "Programacion Orientada a Objetos";
            _docente = "Ing. Torres";
        }
                                    public    override    ExamenPrototype    Clone()    =>
(POOExamen)this.MemberwiseClone();
         public override string VerExamen() => $"{_nombreAsignatura} [{_claveMateria}]
Docente: {_docente} Grupo: {_grupo} Salon: {_salon} Tipo: {_tipoExamen}";
    }
}


namespace PatronPrototipoExamenes
{
    /// Materia: Patrones de Diseño.
    /// HERENCIA: toma todos los atributos de ExamenPrototype.
    /// CLONACION: MemberwiseClone hace una copia exacta de este examen.
    public class PatronesDisenioExamen : ExamenPrototype
    {
        public PatronesDisenioExamen()
        {
            _claveMateria = "TIC-401";
```

```
        _nombreAsignatura = "Patrones de Diseño";
        _docente = "Dr. Ramirez";
    }
    public override ExamenPrototype Clone() => (PatronesDisenioExamen)this.MemberwiseClone();
    public override string VerExamen() => $"{_nombreAsignatura} [{_claveMateria}] Docente: {_docente} Grupo: {_grupo} Salon: {_salon} Tipo: {_tipoExamen}";
  }
}
```

Resultado

```
Seleccionar C:\Users\Nepi-Senpai\source\repos\Actividad de evidencia Individual del patrón Prototipo\Actividad de evidencia Individual del patrón Prototipo\bin\De
Patrones de Diseño [TIC-401] Docente: Dr. Ramirez Grupo: GDS-81A Salon: Aula 301 Tipo: Parcial 1
Patrones de Diseño [TIC-401] Docente: Mtra. Lopez Grupo: GDS-81B Salon: Aula 302 Tipo: Final
Programacion Orientada a Objetos [TIC-302] Docente: Ing. Torres Grupo: GDS-61A Salon: Lab 1 Tipo: Parcial 2
Base de Datos [TIC-305] Docente: Dr. Ramirez Grupo: GDS-61A Salon: Aula 110 Tipo: Parcial 1
Redes de Computadoras [TIC-310] Docente: Mtra. Lopez Grupo: GDS-71A Salon: Lab Redes Tipo: Final
Sistemas Operativos [TIC-308] Docente: Ing. Vega Grupo: GDS-71A Salon: Aula 210 Tipo: Parcial 1
Ingenieria Web [TIC-315] Docente: Mtra. Lopez Grupo: GDS-81D Salon: Lab Web Tipo: Parcial 2
Calculo Diferencial [MAT-101] Docente: Dr. Herrera Grupo: GDS-11A Salon: Aula 101 Tipo: Parcial 1
Inteligencia Artificial [TIC-420] Docente: Dr. Herrera Grupo: GDS-91A Salon: Lab IA Tipo: Final
```

Lo mas importante de la práctica

Clase base ExamenPrototype Es la plantilla general, define los atributos con protected para que nadie los modifique desde afuera, y obliga a todas las materias a tener Clone() y VerExamen().

Herencia Cada materia como POOExamen o IAExamen hereda de ExamenPrototype, lo que significa que ya trae los atributos listos sin repetir código.

Clonación Con MemberwiseClone() se hace una copia exacta del prototipo base y solo se cambia lo que varía, como el grupo, salón o tipo de examen.

Datos protegidos Los atributos con protected como _claveMateria o _docente solo los puede usar la clase base y las que heredan de ella, nadie más.

Conclusión

El patrón Prototype me pareció útil porque en lugar de crear cada examen desde cero, simplemente podemos clonar una base y cambiar lo que necesita. Esto nos ahorra mucho tiempo y evita errores. Los datos protegidos nos dan seguridad de que nadie va a modificar algo que no debe, y gracias a la herencia todas las materias siguen el mismo orden. Es un patrón que nos ayudaría bastante en proyectos reales donde se pueden repetir varias estructuras de código.